

A PRACTITIONER'S DEEP DIVE

SPARK QUERY OPTIMIZATION AND THE ICEBERG LAKEHOUSE

*Why your queries run the way they do — from Catalyst
to the shuffle to the metadata layer — and how to change it*

Written against: Spark 3.5.2 · Apache Iceberg v2 · Kyuubi
Hive Metastore · Google Cloud Storage · Kubernetes

Alan Phan

2026

Preface

[Section to be written.]

Who This Book Is For

[Section to be written.]

How to Use This Book

[Section to be written.]

Conventions Used in This Book

[Section to be written.]

The three callouts look like this:

Note

Marks a version gotcha, a clarification, or a pointer to a later chapter.

Warning

Marks a default or a pattern that silently costs money, correctness, or data.

Tip

Marks a diagnostic shortcut — a UI panel to check, a one-line command.

Contents

Preface	i
Who This Book Is For	ii
How to Use This Book	iii
Conventions Used in This Book	iv
I How Spark Executes a Query	1
1 The Shape of a Spark Application	2
1.1 A query, end to end	2
1.2 Driver, executors, and the cluster manager	3
1.3 Deploy modes and Spark on Kubernetes	5
1.4 Jobs, stages, and tasks	5
1.5 Narrow versus wide dependencies	7
1.6 Reading the UI for the first time	8
1.7 Summary	9
2 From SQL to Execution: the Catalyst Pipeline	10
2.1 The pipeline, overview	10
2.2 Unresolved to resolved: the catalog	11
2.3 Logical optimization: the rules that matter	12
2.4 Physical planning and strategies	13
2.5 Whole-stage code generation	14

2.6	Reading EXPLAIN	14
2.7	Summary	16
3	How an Executor Uses Memory and CPU	18
3.1	The JVM in one page: bytecode and just-in-time compilation	18
3.2	Heap layout and object representation	19
3.3	The executor memory model	20
3.4	Garbage collection: Parallel, G1, ZGC	23
3.5	Serialization and its cost	24
3.6	Summary	25
II	Making Queries Fast	27
4	The Shuffle	28
4.1	What a partition is: input partitions versus shuffle partitions	28
4.2	The shuffle write path	28
4.3	The shuffle read path	29
4.4	Sizing shuffle partitions	29
4.5	Shuffle compression	29
4.6	Push-based shuffle	29
4.7	Finding and removing unnecessary exchanges	29
4.8	Summary	29
5	Joins	30
5.1	Broadcast hash join	30
5.2	Shuffle hash join	30
5.3	Sort-merge join	30
5.4	Broadcast nested loop join	31
5.5	How the planner chooses	31
5.6	Forcing a strategy	31
5.7	Storage-partitioned joins for bucketed Iceberg tables	31
5.8	Summary	31

6	Adaptive Query Execution	32
6.1	Why runtime beats planning time	32
6.2	Coalescing shuffle partitions	32
6.3	Converting sort-merge to broadcast at runtime	32
6.4	The two broadcast thresholds	33
6.5	Skew-join handling: split and replicate	33
6.6	parallelismFirst and partition count	33
6.7	Reading an AQE plan	33
6.8	Summary	33
7	Partitioning, Bucketing, and Pruning	34
7.1	Partition pruning and the pushdown killers	34
7.2	Dynamic partition pruning	34
7.3	Bucketing to avoid a shuffle	35
7.4	repartition versus coalesce	35
7.5	Column pruning and ReadSchema	35
7.6	Summary	35
8	Caching and Reuse	36
8.1	When caching pays	36
8.2	Lazy cache() versus eager CACHE TABLE	36
8.3	Storage levels and the GC trade-off	36
8.4	How execution and storage memory borrow at runtime	37
8.5	Exchange and subquery reuse	37
8.6	Unpersist and cache thrashing	37
8.7	Summary	37
III	The Iceberg Lakehouse	38
9	Table Formats and the Iceberg Spec	39
9.1	From Hive tables to table formats	39
9.2	The Iceberg spec, format version 2	39

9.3	Snapshots and the metadata tree	39
9.4	Position deletes versus equality deletes	40
9.5	Iceberg, Delta Lake, and Hudi	40
9.6	Summary	40
10	Catalogs and Commits	41
10.1	What a catalog does	41
10.2	The Hive Metastore catalog	41
10.3	The REST catalog	41
10.4	The atomic pointer swap	42
10.5	Optimistic concurrency and retries	42
10.6	Catalog configuration in Spark	42
10.7	Summary	42
11	Iceberg Partitioning and Layout	43
11.1	Hidden partitioning and partition transforms	43
11.2	Partition evolution	43
11.3	Sort orders and Z-ordering	43
11.4	Target file size and the small-files problem	44
11.5	Write distribution modes	44
11.6	Summary	44
12	Reading Iceberg Efficiently	45
12.1	Manifest pruning and metadata filtering	45
12.2	Column statistics and predicate pushdown into Iceberg	45
12.3	Metrics modes: full, truncate, counts, none	45
12.4	Bloom filters	46
12.5	Vectorized reads	46
12.6	Split planning	46
12.7	The metadata tables as a diagnostic	46
12.8	Summary	46

13 Writing and Maintaining Iceberg	47
13.1 Copy-on-write versus merge-on-read	47
13.2 MERGE, UPDATE, and DELETE	47
13.3 Write distribution and skew on write	47
13.4 Compacting data files	48
13.5 Rewriting manifests	48
13.6 Snapshot expiration and orphan files	48
13.7 Metadata management for streaming tables	48
13.8 Summary	48
 IV Operating Spark	 49
14 Tuning Resources and Configuration	50
14.1 Executor sizing from the mechanism	50
14.2 Dynamic allocation	50
14.3 Parallelism defaults	51
14.4 The external and push-based shuffle service	51
14.5 Configuration precedence and profiles	51
14.6 Kyuubi and multi-tenancy	51
14.7 Summary	51
 15 Debugging and Profiling	 52
15.1 The Spark UI as a diagnostic	52
15.2 Event logs and the history server	52
15.3 Reading task, shuffle, spill, and GC metrics	52
15.4 Common failure signatures	53
15.5 async-profiler and flame graphs	53
15.6 Debugging query plans	53
15.7 Summary	53
 16 Streaming into the Lakehouse	 54
16.1 The micro-batch model and the unbounded table	54

16.2	Sources and sinks	54
16.3	Event time and watermarking	54
16.4	Stateful operations and the state store	55
16.5	Output modes and triggers	55
16.6	Exactly-once and the commit	55
16.7	Writing to Iceberg: cadence, small files, maintenance	55
16.8	Summary	55
V	Internals	56
17	Tungsten and Code Generation	57
17.1	The Tungsten binary row format	57
17.2	Off-heap memory management	57
17.3	Cache-aware computation	57
17.4	Whole-stage code generation internals	58
17.5	Expression code generation	58
17.6	Columnar processing and vectorization	58
17.7	Summary	58
18	The Shuffle Framework and the Scheduler	59
18.1	The shuffle manager and shuffle writers	59
18.2	The map output tracker	59
18.3	The DAG scheduler and the task scheduler	60
18.4	RPC and block transfer	60
18.5	Locality and speculation	60
18.6	Summary	60
19	DataSource V2 and Extensions	61
19.1	The DataSource V2 read and write API	61
19.2	Pushdown negotiation	61
19.3	How Iceberg reports splits and statistics	61
19.4	Custom Catalyst rules and SQL extensions	62

Contents	xi
19.5 Plugins	62
19.6 Summary	62
A The Reference Environment and the Running Example	63
B Configuration Reference	64
C Reading Query Plans	65
D Glossary	66
E Further Reading	68
References	69

List of Tables

List of Figures

1.1	One application: a single driver process that plans and coordinates, and a set of executor processes that run tasks and cache data. The cluster manager — here the Kubernetes API — hands out the containers the processes run in.	4
1.2	The specimen as one job of three stages. Stage 0 scans and filters trips; stage 1 scans <code>dim_route</code> ; both write shuffle files partitioned by <code>route_id</code> . Stage 2 reads from both, joins, and aggregates. Each stage runs one task per input partition. One task in stage 2 runs far longer than its peers — the first sighting of skew.	6
1.3	A narrow dependency: each output partition is built from one input partition, so the work pipelines in place. A wide dependency: each output partition draws from every input partition, so the data must be redistributed — a shuffle — and the stage ends.	7
2.1	The compilation pipeline. Parsing produces a tree that is pure syntax. Analysis resolves names against the catalog. Logical optimization rewrites the tree with rules. Physical planning picks concrete operators and shuffle points. Code generation emits Java that the JIT compiles. The side notes name what each stage decides.	11
3.1	An executor’s memory: the JVM heap (reserved / unified / user), the optional off-heap pool, and the overhead the process needs outside both. The cluster manager’s container limit must cover all three. Numbers are worked for an 8 GB heap with 2 GB overhead.	22

3.2 A stop-the-world pause freezes every task slot on the executor at once,
not one at a time. If the pause outlasts the heartbeat interval, the driver
concludes the executor is gone and re-runs its work elsewhere — so a
GC problem shows up as lost executors and repeated stages, not just as
slow ones. 24

Part I

How Spark Executes a Query

Chapter 1

The Shape of a Spark Application

A query you would actually write — a week of trips joined to a route dimension, aggregated to an on-time rate — runs in four minutes. This chapter does not make it faster. It uses it as a specimen: you run it, open the Spark UI, and learn to name every moving part — the driver, the executors, the one job, its stages, the shuffle between them, and the one task doing more than the rest. By the end you can read a running job instead of watching a progress bar.

1.1 A query, end to end

Here is the specimen. It asks, for each bus route, what fraction of trips in the first week of March arrived on time. It runs against the `trips` table and the `dim_route` dimension introduced in Appendix A; `trips` is an Iceberg table partitioned by `day(event_ts)`, holding a few hundred million rows for the month.

```
1 SELECT r.route_name ,
2         AVG(CASE WHEN t.arrival_delay_s <= 120 THEN 1.0 ELSE 0.0
3             END) AS on_time_rate
4 FROM trips t
5 JOIN dim_route r ON t.route_id = r.route_id
6 WHERE t.event_ts >= TIMESTAMP '2026-03-01 00:00:00'
7        AND t.event_ts <  TIMESTAMP '2026-03-08 00:00:00'
```

```
7 GROUP BY r.route_name  
8 ORDER BY on_time_rate;
```

Listing 1.1: The specimen query

Submit it and Spark does not simply “run the SQL.” The text is parsed into a tree, resolved against the catalog, rewritten by the optimizer, turned into a plan of physical operators, and compiled to Java bytecode. That whole compilation is Chapter 2’s subject; for now, take it as given that by the time anything executes, Spark holds a *physical plan*: a tree of operators such as *scan*, *filter*, *exchange* (a shuffle), *sort-merge join*, and *hash aggregate*.

Executing that plan is a distributed affair. The plan is split into a *job*, the job into *stages*, and each stage into *tasks* that run in parallel on *executors*. The result flows back to the process that submitted the query, the *driver*. On the reference cluster this specimen produces one job, three stages, and a few hundred tasks, and it takes about four minutes.

Why four minutes and not one, or forty? That question is what the rest of the book answers. This chapter’s job is narrower: to give you the vocabulary and the tools to *see* the structure of the run, so that later chapters have something concrete to change. Everything that follows — joins, the shuffle, adaptive execution, the Iceberg metadata layer — is a way of moving less data, moving it less often, or moving it more evenly. You cannot decide to move data differently until you can see where it moves now.

1.2 Driver, executors, and the cluster manager

A Spark *application* is one driver plus a set of executors, all working on your queries until the application exits (Figure 1.1).

The **driver** is the process that runs your program. It holds the `SparkSession`, compiles each query into a plan, breaks the plan into stages and tasks, hands tasks to executors, tracks their progress, and gathers results. It is a single process and a single point of failure: if the driver dies, the application dies. It is also where a `collect()` or a `toPandas()` lands every row it pulls back, which is the most common way to run the driver out of memory (Chapter 3, Section 15.3).

An **executor** is a JVM process that does the actual work. Each executor has a fixed

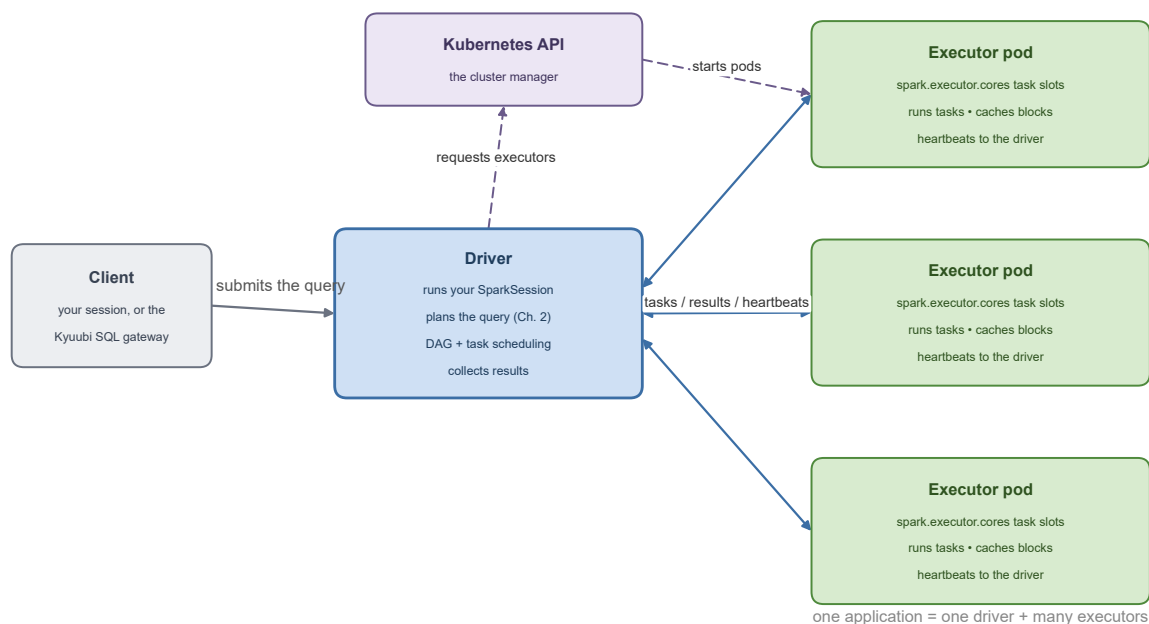


Figure 1.1: One application: a single driver process that plans and coordinates, and a set of executor processes that run tasks and cache data. The cluster manager — here the Kubernetes API — hands out the containers the processes run in.

number of *task slots* — one per core, set by `spark.executor.cores` — and runs one task per slot at a time. Executors also hold cached blocks and the map-side output of shuffles, and they send regular heartbeats to the driver so it knows they are alive. An application typically has many executors and exactly one driver.

The **cluster manager** owns the machines and hands out the containers that the driver and executors run in. Spark supports several: its own standalone manager, Hadoop YARN, and Kubernetes (Apache Mesos support was removed in Spark 3.2). The reference stack uses Kubernetes, so “container” means “pod” and the cluster manager is the Kubernetes API server.

Note

The driver–executor split is why two very different failures look different. An executor lost to an out-of-memory kill takes its tasks down, and Spark simply re-runs them elsewhere; the job slows but survives. The driver running out of memory — usually from pulling too much data back — kills the whole application at once, with no retry.

1.3 Deploy modes and Spark on Kubernetes

Where the *driver* runs is the *deploy mode*. In **client mode** the driver runs in whatever process launched the application — your shell, a notebook kernel, or a long-lived gateway. In **cluster mode** the launcher asks the cluster manager to start the driver as just another container, then disconnects. Client mode is convenient for interactive work because the driver is local to you; cluster mode is standard for scheduled jobs because nothing on your side has to stay running.

On Kubernetes the sequence for a cluster-mode job is: `spark-submit` calls the Kubernetes API to create a *driver pod*; the driver pod starts, and its Spark code calls the same API to create *executor pods*; each executor pod starts, connects back to the driver, and registers its task slots. From then on the driver schedules tasks onto executors directly, and the Kubernetes API is only involved again when pods are added or removed — for example under dynamic allocation (Section 14.1).

The reference stack does not usually run `spark-submit` per query. **Kyuubi** runs a long-lived driver (a persistent `SparkSession`, one per user or group) and you send SQL to it over a connection, the way you would to any database. The engine, stages, and tasks are all still there; Kyuubi just keeps the driver warm so you are not paying startup cost on every query and so many users can share a pool of executors.

1.4 Jobs, stages, and tasks

Spark transformations are *lazy*: `filter`, `join`, `groupBy`, and the rest build up a plan but run nothing. Work starts only when an *action* asks for a result — a write, a count, a collect, or the implicit collect that returns rows to a SQL client. Each action triggers one **job**.

Within a job, the driver's *DAG scheduler* cuts the plan into **stages**. The cut always falls at a shuffle: a stage is the longest run of work that can be pipelined without moving data between executors. Everything up to a shuffle goes in one stage; the shuffle ends it; what comes after starts a new one. A stage that reads a shuffle cannot begin until the stage that writes it has finished, which is why stage boundaries are also the points where

a job can stall.

A stage is executed as a set of **tasks** — one per partition of the stage’s input, each doing identical work on its own slice of the data. A task is the unit Spark schedules onto an executor’s task slot; one task occupies one core for its duration. The number of tasks in a stage is therefore the number of partitions, and it sets how much parallelism the stage can use: 8 tasks on a cluster with 200 free slots leaves 192 idle.

Figure 1.2 shows the specimen’s shape. Because `dim_route` has no statistics that would let the planner broadcast it (an Iceberg table usually does not, until you `ANALYZE` it — Chapter 5), the planner chooses a sort-merge join: both inputs are shuffled so that rows with the same `route_id` land on the same executor. That gives:

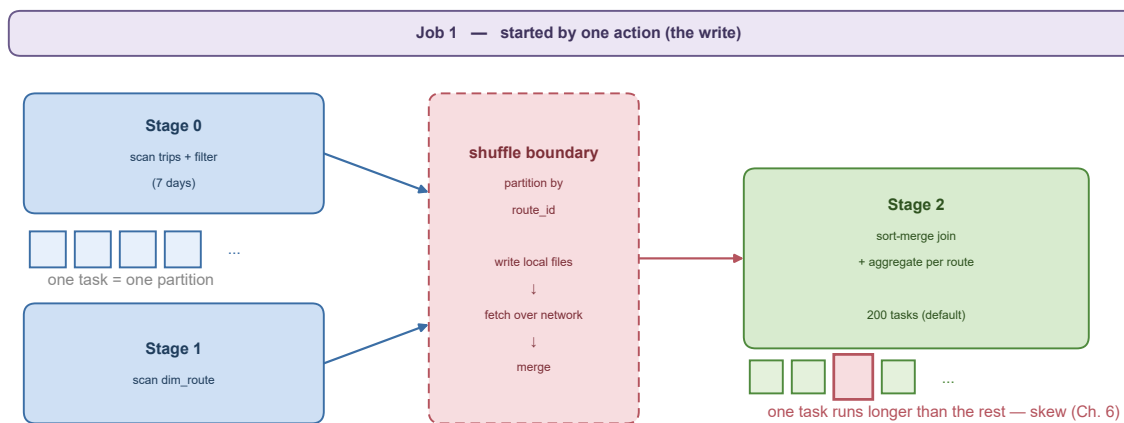


Figure 1.2: The specimen as one job of three stages. Stage 0 scans and filters trips; stage 1 scans `dim_route`; both write shuffle files partitioned by `route_id`. Stage 2 reads from both, joins, and aggregates. Each stage runs one task per input partition. One task in stage 2 runs far longer than its peers — the first sighting of skew.

- **Stage 0** — scan the March partitions of `trips`, apply the timestamp filter, and write shuffle files keyed by `route_id`. Roughly 180 tasks, one per input file split.
- **Stage 1** — scan `dim_route` and write shuffle files keyed by `route_id`. A handful of tasks; the table is tiny.

- **Stage 2** — fetch both sides' shuffle files, merge them on `route_id`, join, and aggregate per route. 200 tasks, because `spark.sql.shuffle.partitions` defaults to 200 (Chapter 4). The final `ORDER BY` is small enough to finish inside this stage.

The `ORDER BY` on the aggregated result is cheap — a few thousand rows — so it does not add a stage here. On a larger result it would.

1.5 Narrow versus wide dependencies

The reason a shuffle ends a stage is the kind of dependency it creates between partitions (Figure 1.3).

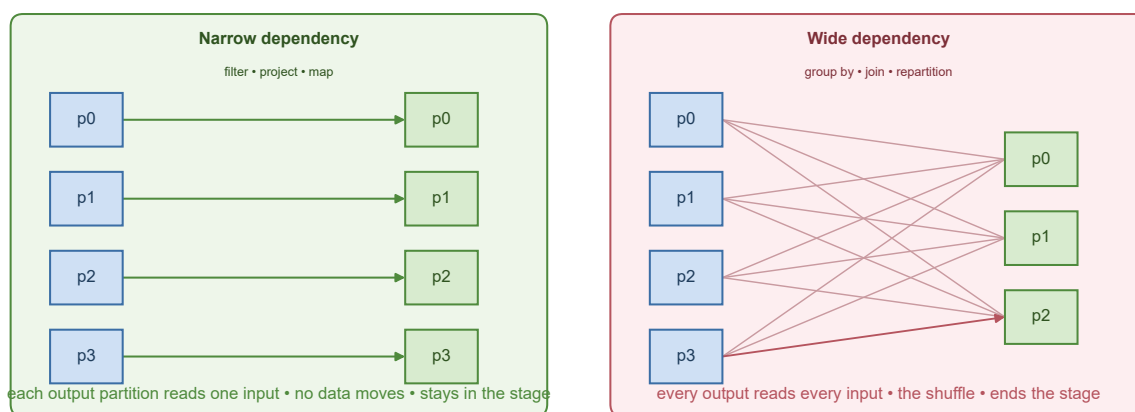


Figure 1.3: A narrow dependency: each output partition is built from one input partition, so the work pipelines in place. A wide dependency: each output partition draws from every input partition, so the data must be redistributed — a shuffle — and the stage ends.

A **narrow** dependency is one where each output partition depends on a bounded, known set of input partitions — usually exactly one. `filter`, `select`, and `map` are narrow: partition 3 of the output is partition 3 of the input with a function applied. Narrow operations chain together with no data movement, which is why an entire run of them collapses into a single stage and executes as one pipelined pass per partition.

A **wide** dependency is one where an output partition can depend on *every* input partition. `groupBy`, `join` (unless one side is broadcast), `distinct`, `repartition`, and

window functions are wide: to put all rows for `route_id = 47` in one place, Spark must look at every input partition, because any of them might hold such a row. That redistribution is the shuffle, and it is the wall between stages.

This is the single most useful idea in the book: **cost lives at stage boundaries**. Narrow work is cheap and scales cleanly. Wide work writes data to disk, moves it over the network, and re-reads it, and it forces the whole cluster to wait at a barrier. Almost every technique in Chapter 4 through Chapter 8 is about having fewer wide dependencies, making the ones you keep move less data, or making them move it more evenly.

1.6 Reading the UI for the first time

The Spark UI — served by the driver, on port 4040 by default, and preserved after the fact by the history server (Section 15.3) — has a tab for each level of the model.

SQL / DataFrame The query as a plan tree, annotated with the number of rows and the time spent at each operator after execution. This is where you confirm which join strategy ran and count the *Exchange* nodes; each Exchange is one shuffle.

Jobs One row per action. The specimen shows a single job.

Stages The stages of each job, with a timeline and, for every stage, a *Summary Metrics* table: the minimum, 25th percentile, median, 75th percentile, and maximum across the stage’s tasks for duration, shuffle read size, records, spill, and GC time.

Executors One row per executor: task slots, memory used, disk spilled, and total GC time. A cluster where one executor shows far more GC than the others is usually holding skewed data.

Storage Cached datasets and how much of each is actually resident.

For the specimen, open the Stages tab and look at stage 2’s Summary Metrics. The median task duration is a few seconds; the maximum is close to a minute. One task is doing a large multiple of the work of the typical task. That is *data skew*: one `route_id` — a busy trunk route — holds far more trips than the others, so the task that got that route’s partition runs alone while the rest of the stage has finished. You cannot fix it yet.

Chapter 6 and Chapter 5 will. But you can now *see* it, and seeing it is the prerequisite for everything else.

Tip

Four numbers tell you most of what is wrong with a slow stage: max versus median *task duration* (skew), *shuffle read/write* size (how much data the wall is moving), *spill* (not enough execution memory, Chapter 3), and *GC time* (too much, Section 3.4). Check them in that order.

1.7 Summary

You can now describe what happens when a query runs. The *driver* plans it and coordinates; *executors* run *tasks*, one per core, on partitions of the data; the *cluster manager* (Kubernetes here) provides the containers. An action triggers a *job*; the job is cut into *stages* at every shuffle; a stage runs one task per partition. A *narrow* dependency keeps work in place; a *wide* dependency forces a shuffle and ends the stage, and that is where a query spends most of its time. In the Spark UI you can find the job, its stages, and their tasks, identify a stage boundary as a shuffle from the Exchange nodes in the SQL tab, and spot an uneven stage from the gap between its maximum and median task time.

The specimen still takes four minutes. Chapter 2 opens up how Spark decided on this plan; Chapter 4 onward is how you change it.

Exercises

1. Run two queries against `trips`: `SELECT count(*) FROM trips` and `SELECT route_id, count(*) FROM trips GROUP BY route_id`. Open the Stages tab for each. Which produced a second stage, and why?
2. In the SQL tab for the specimen query, count the *Exchange* nodes. Match each one to a stage boundary in Figure 1.2.
3. In stage 2's Summary Metrics, record the median and maximum task duration and the maximum shuffle-read size. Which of the four numbers from the tip is the outlier?

Chapter 2

From SQL to Execution: the Catalyst Pipeline

A SQL string does not run. It is parsed into a tree, resolved against the catalog, rewritten by rules, turned into a physical plan by choosing strategies, and compiled to JVM bytecode. Every change you can make to a query's performance is a change to one of those stages. This chapter traces the specimen query from Chapter 1 through all of them and teaches you to read `EXPLAIN FORMATTED` well enough to predict the physical plan before you run it.

2.1 The pipeline, overview

Catalyst is the optimizer inside Spark SQL. It is a library for building and rewriting trees, plus a fixed sequence of passes that turn a query into an executable plan (Figure 2.1). Every DataFrame operation and every SQL statement goes through the same pipeline; the DataFrame API is just a second front end that builds the same tree the SQL parser does.

Four representations matter, and you can print all of them (Section 2.6):

1. the **unresolved logical plan** — the parse tree; relations and columns are just names.

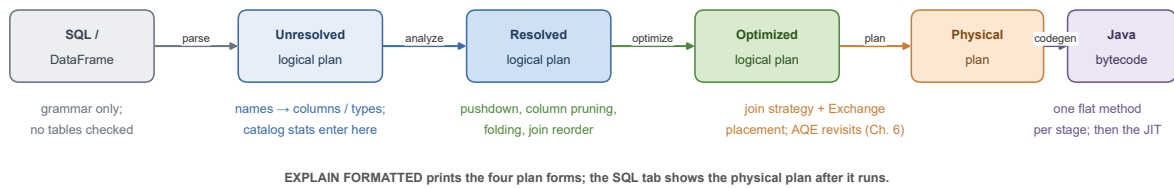


Figure 2.1: The compilation pipeline. Parsing produces a tree that is pure syntax. Analysis resolves names against the catalog. Logical optimization rewrites the tree with rules. Physical planning picks concrete operators and shuffle points. Code generation emits Java that the JIT compiles. The side notes name what each stage decides.

2. the **resolved (analyzed) logical plan** — names bound to real tables and typed columns.
3. the **optimized logical plan** — the same query, rewritten to do less work.
4. the **physical plan** — a tree of executable operators, with the shuffles made explicit as *Exchange* nodes.

The physical plan is what Chapter 1 called “the plan.” Reading it, and knowing which pass produced each part of it, is what lets you look at a query and say in advance how many stages it will have and where they will stall.

2.2 Unresolved to resolved: the catalog

The parser only checks grammar. `SELECT r.route_name FROM trips t JOIN dim_route r ...` parses whether or not those tables exist; the result is a tree whose relation and attribute nodes are still marked *unresolved* — `trips` and `r.route_name` are held as names, not yet bound to anything.

The *analyzer* resolves them. It asks the **catalog** what `trips` and `dim_route` are, gets back their schemas, and replaces each unresolved node with a concrete one: `route_name` becomes a `string` column belonging to `dim_route`, `arrival_delay_s` becomes an `int` on `trips`, the `JOIN` condition is type-checked, and `AVG(...)` is bound to the aggregate

function. If a name does not resolve, this is where the query fails — before any work is scheduled.

In the reference stack the catalog is a Spark *catalog plugin* pointed at an Iceberg REST endpoint (backed by the Hive Metastore); Chapter 10 covers what it stores and how commits move through it. What matters here is that the catalog is also where *table statistics* come from — row counts, column value ranges, distinct counts. Spark’s cost-based optimizer uses them to order joins and to decide what is small enough to broadcast (Chapter 5). An Iceberg table exposes a total file size and row count from its metadata, but not per-column statistics unless you have run `ANALYZE TABLE ...COMPUTE STATISTICS`. This is why so many lakehouse joins pick the wrong strategy at planning time and have to be rescued by a hint or by AQE.

2.3 Logical optimization: the rules that matter

The optimizer applies *rules* — functions from a logical plan to a better logical plan — grouped into *batches* that run repeatedly until the plan stops changing. Spark ships around a hundred rules. A handful account for most of the speed-up, and all of them do the same thing: move work earlier, or delete it.

Predicate pushdown. A filter is pushed as close to the scan as the semantics allow, so fewer rows enter the rest of the plan. In the specimen the `event_ts` range moves into the `trips` scan, where it becomes a *partition filter* (Iceberg skips whole days) and a *pushed filter* (Parquet skips row groups). Chapter 7 is about keeping this rule working; a function or a `CAST` on the filtered column defeats it.

Column pruning. The scan reads only the columns the query references. The specimen touches `route_id`, `arrival_delay_s`, and `event_ts` from `trips` and `route_id`, `route_name` from `dim_route` — so a fifty-column `trips` table is read five columns wide. A `SELECT *` anywhere in a subquery turns this off.

Constant folding and simplification. Expressions with no runtime input are evaluated once at planning time; the `TIMESTAMP '2026-03-01 ...'` literals are computed here, not

per row. Boolean and null simplifications (`x AND true`, `x = x`, `IS NOT NULL` on a non-nullable column) are folded away.

Join reordering. With reliable statistics and cost-based optimization enabled (`spark.sql.cbo.enabled` and `spark.sql.cbo.joinReorder.enabled`), a multi-way join is reordered to keep intermediate results small — join the pair that produces the fewest rows first. Without statistics Spark keeps the written order, which is one more reason Iceberg tables benefit from `ANALYZE`.

Note

Most optimizer rules are *heuristic*: they rewrite the plan into a form that is almost always better, using no cost model. Only join selection and join reordering are cost-based, and only when statistics exist. When people say “the optimizer is smart,” they usually mean the heuristic rules, which are reliable; the cost-based part is only as good as the catalog’s numbers.

2.4 Physical planning and strategies

The optimized *logical* plan still says only *what* to compute. The planner turns each logical operator into one or more *physical* operators by applying *strategies*:

- a logical Join becomes BroadcastHashJoin, ShuffledHashJoin, or SortMergeJoin, chosen from the estimated sizes of its inputs and `spark.sql.autoBroadcastJoinThreshold` (Chapter 5).
- a logical Aggregate becomes a partial HashAggregate on each partition, an Exchange to bring equal keys together, and a final HashAggregate.
- a logical Sort that needs a global order becomes a range-partitioning Exchange followed by a local Sort.

The planner also inserts Exchange (shuffle) and Sort nodes wherever an operator’s required input distribution is not already satisfied — and, crucially, *removes* them where it is. If two joins share a key and the data is already partitioned by it, the second join reuses the first’s exchange. This is why the *number of Exchange nodes in the physical*

plan is the single best predictor of a query’s cost, and why Chapter 1 told you to count them in the UI.

For the specimen, with no broadcastable statistics on `dim_route`, the planner produces: two scans, each followed by an `Exchange hashpartitioning(route_id, 200)`, a `SortMergeJoin`, a partial and final `HashAggregate` (which need no new exchange, because the join already partitioned by `route_id`), and a final small sort. Three stages, exactly as Figure 1.2 showed.

2.5 Whole-stage code generation

A physical plan could be executed by walking the operator tree and calling `next()` on each node per row — the “Volcano” model. Spark does not. For a chain of operators within a stage it *generates Java source* for the whole chain, one method that scans, filters, projects, and feeds the aggregate without a function call or an intermediate row object per step. That method is compiled to bytecode and then by the JVM’s JIT to native code (Chapter 17).

In a printed plan you see this as a numbered marker: operators fused into one generated function are wrapped in `*(2) HashAggregate`, `*(2) Project`, `*(2) Filter`, and so on — the `*(n)` means “part of generated stage *n*.” An operator without a star (a plain `Exchange`, or a source that does not support codegen) is a boundary where Spark falls back to row-at-a-time. A plan that is missing stars where you expect them — often because a UDF or an unusual expression broke the chain — is a plan leaving performance on the table.

2.6 Reading EXPLAIN

EXPLAIN prints the plans without running the query. The default form is terse; EXPLAIN FORMATTED is the one to learn.

```
1 EXPLAIN FORMATTED
2 SELECT r.route_name, AVG(...) FROM trips t JOIN dim_route r ON
   ... WHERE ... GROUP BY ...;
```

Listing 2.1: Printing the plans

The output has two parts: a compact tree, then a numbered list where each node's details are spelled out. Read the tree bottom-up — leaves are scans, the root is the final operator — and check five things.

```

1 == Physical Plan ==
2 * Sort (13)
3 +- * HashAggregate (12)                                <- final
   aggregate, in generated stage
4   +- * HashAggregate (10)                                <- partial
      aggregate, same stage
5   +- * SortMergeJoin Inner (9)                            <- NOT a
      broadcast: both sides shuffled
6     :- * Sort (5)
7       : +- Exchange hashpartitioning(route_id, 200) (4)
          <- shuffle #1
8       :   +- * Filter (3)
9       :     +- * ColumnarToRow (2)
10      :       +- BatchScan trips (1)
11     +- * Sort (8)
12       +- Exchange hashpartitioning(route_id, 200) (7)
          <- shuffle #2
13       +- BatchScan dim_route (6)
14
15 (1) BatchScan trips
16   PartitionFilters: [event_ts >= 2026-03-01, event_ts <
   2026-03-08]    <- days pruned
17   PushedFilters:   [IsNotNull(route_id)]
                                   <- row groups skipped
18   ReadSchema:      struct<route_id,arrival_delay_s,event_ts>
                                   <- only 3 columns

```

Listing 2.2: The parts of EXPLAIN FORMATTED that matter (trimmed)

1. **Join operator.** SortMergeJoin means both sides were shuffled; BroadcastHashJoin means one side was small enough to ship whole. If you see the former on an obviously small dimension, the planner lacked statistics — Chapter 5.
2. **Exchange nodes.** Count them. Each is a stage boundary and a shuffle. Two here.
3. **PartitionFilters.** Non-empty means Iceberg will skip whole partitions. Empty on a partitioned table means the filter did not match the partition transform — Chapter 7.
4. **PushedFilters.** Non-empty means Parquet row groups will be skipped inside the files that are read.
5. **ReadSchema.** Should list only the columns the query needs. A wide schema here means a `SELECT *` leaked in.

`EXPLAIN COST` additionally prints the row-count and size estimates the planner used — useful for seeing *why* it did not broadcast. After the query runs, the SQL tab shows the same tree with actual row counts and, if AQE was active, the final adapted plan marked `AdaptiveSparkPlan isFinalPlan=true` (Chapter 6).

2.7 Summary

A query becomes an execution in fixed stages: parse to a syntax tree, resolve names and types against the catalog, rewrite the logical plan with mostly heuristic rules (pushdown, column pruning, folding; join reordering only with statistics), choose physical operators and shuffle points with strategies, and generate Java for each pipelined chain. The physical plan — readable with `EXPLAIN FORMATTED` — tells you the join strategy, the number of shuffles, and whether pruning and column selection are working, which together determine how many stages the query has and where it will spend its time. The recurring lakehouse failure is missing catalog statistics, which makes the cost-based decisions (join strategy and order) unreliable until you `ANALYZE` or override.

Exercises

1. Run `EXPLAIN FORMATTED` on the specimen query. Confirm the two Exchange nodes, the SortMergeJoin, and a non-empty PartitionFilters on the trips scan.
2. Write the same result three ways: with an explicit subquery, with a CTE, and as one flat statement. Compare the *optimized* logical plans. Which differ, and which are identical?
3. Add `WHERE CAST(t.route_id AS STRING) = '47'` to the query and re-run `EXPLAIN FORMATTED`. What happened to PushedFilters?

Chapter 3

How an Executor Uses Memory and CPU

Spark runs on the Java Virtual Machine: the driver and every executor is a JVM process, and the engine's memory behaviour, its failure modes, and much of its internal design follow directly from how the JVM manages memory and runs code. This chapter builds the memory map of a single executor — which regions exist, which a query can use, and which are consumed no matter what — then adds the two costs layered on top of it: garbage collection and serialization. By the end you can size a pod from its configuration, explain a container kill that left no Java error, and read GC time in the UI. How execution and storage memory borrow from each other while a job runs is picked up alongside caching in Chapter 8; how to size these regions for a workload is Chapter 14.

3.1 The JVM in one page: bytecode and just-in-time compilation

Spark is written in Scala and runs on the Java Virtual Machine, and two features of that platform shape how it performs.

The first is *just-in-time compilation*. Scala compiles to JVM *bytecode*, a portable instruction set the JVM does not run directly. Instead it interprets bytecode at first and

watches which methods run often; a *hot* method is then compiled to native machine code by the JIT, with optimizations — inlining, loop unrolling, dead-code elimination, escape analysis — that depend on what the method actually did the first few thousand times it ran. Two consequences follow. A Spark process is slowest in its first seconds, before the JIT has warmed up, which is one reason Kyuubi keeps a driver alive rather than starting one per query (Section 1.3) and why a micro-benchmark that runs a snippet once tells you almost nothing. And the JIT compiles *tight, predictable* code far better than code full of virtual calls and branches — which is exactly why Spark’s execution engine generates a single flat Java method for a chain of operators (Section 2.5, Chapter 17) instead of walking an operator tree.

The second is *automatic memory management*: the program allocates objects and never frees them, and a *garbage collector* reclaims what is no longer reachable. This is a convenience with a running cost, and for a data engine moving billions of rows the cost is large enough that Spark works hard to keep most of its data *out* of the managed heap entirely. The rest of this chapter is about where an executor’s memory goes, and Section 3.4 about what the garbage collector does to it.

3.2 Heap layout and object representation

The JVM manages a single contiguous region of memory, the *heap*, from which every object a program allocates is drawn. A Spark executor is one JVM process, and the value of `spark.executor.memory` is precisely the maximum size of that heap. Everything a task does — decoding a Parquet page into rows, building a hash table for a join, accumulating a group-by result — competes for space in it, and when it fills, the task must either spill to disk or fail.

Two properties of the heap drive most of Spark’s decisions about memory. The first is *per-object overhead*. Every object carries a header — a mark word and a pointer to its class — before any field is stored, and the allocator pads every object to an eight-byte boundary.

```
java.lang.Integer
  object header    12 bytes    (mark word + class pointer)
  int value        4 bytes
```

```

-----
total          16 bytes    to carry 4 bytes of data

```

Listing 3.1: Cost of a boxed integer on a 64-bit JVM

A row represented as an array of boxed Java objects (an `Integer`, a `Double`, a `String` with its own header and backing character array) can be several times larger in the heap than the same row encoded as raw bytes. This is the amplification you see when a compressed Parquet file expands to many times its on-disk size once decoded into Java objects, and it is why Spark’s execution engine stores rows off the object heap in a packed binary format rather than as JVM objects (Chapter 17).

The second property is that the heap is *managed*: the program never frees memory explicitly, and reclamation is the job of the garbage collector (Section 3.4). Spark does not hand the whole heap to a query; it subdivides it into regions with different owners and different rules, described next.

3.3 The executor memory model

Configuring Spark’s memory means deciding how one executor process divides the memory it is allowed to use. That process holds three separately configured pools, and confusing them is the most common source of both wasted memory and unexplained container kills.

The heap. `spark.executor.memory` sets the maximum size of the JVM heap, the region every object the executor allocates comes from. Spark carves it into three parts:

- **Reserved memory** — a fixed 300 MB Spark holds back for its own internal objects. It is not configurable and never available to a query.
- **Unified memory** — by default 60% of (heap – 300 MB), set by `spark.memory.fraction`. This is the only pool Spark accounts for byte by byte and the only one that spills to disk gracefully when it runs out. It is shared between *execution memory* (the working memory of a running operator: hash tables for joins and aggregations, sort buffers, the shuffle write buffer) and *storage memory* (cached

DataFrame and RDD blocks, and broadcast data held on the executor). `spark.memory.storageFraction` (default 0.5) is a soft boundary between the two: either side may borrow the other's idle space, but execution can always evict borrowed cached blocks to reclaim its half, whereas storage cannot push execution below the border. How this plays out while a job runs is covered in Chapter 8.

- **User memory** — the remaining 40%. Everything Spark does not manage lives here: the objects your UDFs and RDD closures create, the structures a `mapPartitions` builds up. Spark does not track this pool and cannot spill it; overflowing it throws `OutOfMemoryError` and kills the executor.

Off-heap memory. When `spark.memory.offHeap.enabled` is set, `spark.memory.offHeap.size` adds a second unified-memory pool that lives outside the heap, addressed directly by the engine and invisible to the garbage collector (Section 3.4). Execution and storage draw on it in addition to their on-heap pool. It is not taken out of `spark.executor.memory`; it is additional memory the process reserves, and it must be counted into the container's limit.

Overhead. `spark.executor.memoryOverhead` (default the larger of 384 MB and 10% of the heap) is memory the process needs that is neither heap nor off-heap pool: JVM thread stacks, class metadata (Metaspace), buffers for network transfer and compression codecs, and — for PySpark — the Python worker processes, which can each take hundreds of megabytes. It is not a safety margin; it is space that is actually consumed, and its default is frequently too small for PySpark or codec-heavy workloads, in which case the container is killed with no Spark-level error.

What the cluster manager enforces. The pod or container is sized as the sum of all three pools (Figure 3.1). If the process's real usage crosses that limit — most often because overhead was underestimated or user memory grew unbounded — the kernel kills it (Section 15.4). The heap merely filling up, by contrast, makes unified memory spill to disk and only throws when user memory or a single un-spillable allocation is the cause.



Figure 3.1: An executor’s memory: the JVM heap (reserved / unified / user), the optional off-heap pool, and the overhead the process needs outside both. The cluster manager’s container limit must cover all three. Numbers are worked for an 8 GB heap with 2 GB overhead.

Per task, not per executor. Execution memory is shared equally across the tasks an executor runs at once, which is `spark.executor.cores`. A larger heap with proportionally more cores gives no single task more room: memory per task is the execution pool divided by the core count. The number of cores per executor is therefore a memory decision as much as a parallelism one, and an executor with too many cores spills on work a narrower one finishes in memory (Section 14.1).

3.4 Garbage collection: Parallel, G1, ZGC

Because the heap is managed, the JVM must periodically find objects that are no longer reachable and reclaim their space. This work is the *garbage collector* (GC), and for a data-processing workload it is never free: the collector competes with the application for CPU, and some of its phases are *stop-the-world*, meaning every application thread in the JVM is frozen until the phase completes. On an executor a long stop-the-world pause stalls every running task at once, and if it exceeds the heartbeat timeout the driver declares the executor lost and re-runs its work elsewhere (Section 15.4).

Collectors differ mainly in how they trade pause length against throughput and CPU cost.

- **Parallel GC** uses multiple threads but collects the whole heap in one stop-the-world pause. It gives the highest raw throughput when pauses do not matter, but pause time grows with heap size, so a large executor can freeze for seconds at a time.
- **G1** (Garbage-First) divides the heap into equal-sized regions and collects a subset of them per cycle, doing most of its marking concurrently with the application. Pause times are shorter and roughly bounded regardless of heap size, at the cost of more bookkeeping and CPU. It is the default at the heap sizes a Spark executor typically runs and the collector most Spark tuning assumes.
- **ZGC** does essentially all of its work concurrently and keeps pauses under a millisecond even on very large heaps, at a higher constant CPU and memory overhead. It is worth choosing only for executors with tens of gigabytes of heap where G1

pauses are still a problem.

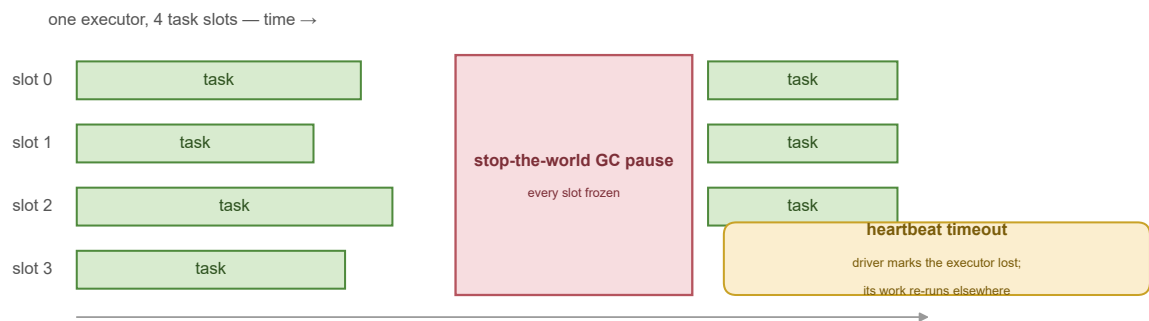


Figure 3.2: A stop-the-world pause freezes every task slot on the executor at once, not one at a time. If the pause outlasts the heartbeat interval, the driver concludes the executor is gone and re-runs its work elsewhere — so a GC problem shows up as lost executors and repeated stages, not just as slow ones.

The generational hypothesis — that most objects die young — underlies all three: they collect a small “young” area cheaply and often, and the full-heap collection rarely. A workload that holds a large amount of long-lived data in cache violates that assumption and pushes the collector toward the expensive path, which is one more reason to cache deliberately (Chapter 8). Monitoring GC time is covered in Section 15.3; the executor JVM options that select and tune a collector are in Chapter 14.

3.5 Serialization and its cost

The engine keeps a join’s hash table or a block of cached rows as one large off-heap byte buffer rather than millions of small Java objects, which is why that data is invisible to the garbage collector and costs nothing to collect (Section 3.3, Chapter 17). Getting data into and out of that flat form is serialization.

Serialization is the conversion of an in-memory object graph into a flat sequence of bytes, and *deserialization* is its inverse. Spark serializes whenever data leaves a JVM: across the network during a shuffle, to disk when unified memory spills or a block is

cached in serialized form, and when a broadcast variable is shipped to executors. The cost is both CPU and size.

The two options for arbitrary Java objects differ sharply. The built-in Java serializer writes the full class name and every field name and type alongside the values, repeated for every object, so most of the output is self-description rather than data. *Kryo* writes a compact encoding — a short numeric class identifier and the field values — and produces output several times smaller, which directly reduces shuffle volume, spill size, and broadcast footprint (Chapter 14).

`DataFrame` and SQL workloads sidestep both serializers for most operations: rows already live in the engine’s binary format (Chapter 17), so moving them is a memory copy with no object graph to walk. The Java-versus-Kryo choice still governs RDD code, user-defined functions, and anything that puts arbitrary Java objects into a shuffle or a broadcast — the same user-memory objects that Section 3.3 noted Spark does not manage.

3.6 Summary

An executor is one JVM process, and its memory is three separately configured pools: the heap (`spark.executor.memory`), an optional off-heap pool, and overhead. The heap is carved into a fixed 300 MB reserve, *unified memory* (the only pool Spark accounts for and the only one that spills), and *user memory* (untracked, un-spillable, throws on overflow). The container limit the cluster manager enforces is the sum of all three; cross it and the process is killed with no Java error, whereas merely filling the heap makes unified memory spill. Execution memory is shared across an executor’s cores, so cores-per-executor is a memory decision. On top of the map sit two costs: garbage collection, whose stop-the-world pauses freeze every task at once and can cost you an executor; and serialization, which Kryo makes several times cheaper than the Java default for RDD and UDF code, though `DataFrame` rows bypass it entirely. You can now size an executor from its workload and explain the two kinds of memory failure by which pool overflowed.

Exercises

1. Given a pod memory limit and a workload description, propose `spark.executor.m`

emory, `spark.executor.cores`, and `spark.executor.memoryOverhead`, and justify each number from the memory map in Figure 3.1.

2. An executor is killed by Kubernetes with `OOMKilled` and no Spark exception in the log. A second run fails with a `Java OutOfMemoryError` in a task. Which pool overflowed in each case, and which knob addresses each?
3. In the Executors tab, one executor shows 40% of task time in GC while the others show 5%. What is the most likely cause, and which chapter covers the fix?

Part II

Making Queries Fast

Chapter 4

The Shuffle

Every JOIN, GROUP BY, DISTINCT, window function, and repartition puts a wall between two stages: the shuffle. It writes every partition to disk, moves it across the network, and re-reads it. It is where most of a query's time goes. This chapter walks the write path and the read path, shows how to size shuffle partitions from data volume instead of accepting the default 200, and how to find and remove exchanges the query does not need.

4.1 What a partition is: input partitions versus shuffle partitions

[Section to be written.]

4.2 The shuffle write path

[Section to be written.]

4.3 The shuffle read path

[Section to be written.]

4.4 Sizing shuffle partitions

[Section to be written.]

4.5 Shuffle compression

[Section to be written.]

4.6 Push-based shuffle

[Section to be written.]

4.7 Finding and removing unnecessary exchanges

[Section to be written.]

4.8 Summary

[Section to be written.]

Exercises

1. From a stage's shuffle-read size in the UI, compute a target partition count for a 128 MB task size, then verify against the observed task sizes.

Chapter 5

Joins

Spark has four ways to run a join and picks one from a size estimate. When the estimate is right, the choice is right. When the table has no statistics — the common case for Iceberg — the planner guesses, and a join that should have broadcast a 30 MB dimension instead shuffles a billion rows. This chapter covers the four strategies, how the planner chooses, and how to override it when you know better than the stats do.

5.1 Broadcast hash join

[Section to be written.]

5.2 Shuffle hash join

[Section to be written.]

5.3 Sort-merge join

[Section to be written.]

5.4 Broadcast nested loop join

[Section to be written.]

5.5 How the planner chooses

[Section to be written.]

5.6 Forcing a strategy

[Section to be written.]

5.7 Storage-partitioned joins for bucketed Iceberg tables

[Section to be written.]

5.8 Summary

[Section to be written.]

Exercises

1. Given four table sizes and the broadcast threshold, predict each join's strategy. Then change one config value and predict which join flips.

Chapter 6

Adaptive Query Execution

The Catalyst optimizer plans before it has seen any data. Adaptive Query Execution re-plans after each shuffle, when the real partition sizes are known. It coalesces hundreds of tiny partitions into a few good ones, promotes a sort-merge join to a broadcast when the shuffled side turns out small, and splits a skewed partition so one task stops holding up the stage. This chapter covers what AQE re-decides, from what evidence, and which planning-time decisions it can still override.

6.1 Why runtime beats planning time

[Section to be written.]

6.2 Coalescing shuffle partitions

[Section to be written.]

6.3 Converting sort-merge to broadcast at runtime

[Section to be written.]

6.4 The two broadcast thresholds

[Section to be written.]

6.5 Skew-join handling: split and replicate

[Section to be written.]

6.6 parallelismFirst and partition count

[Section to be written.]

6.7 Reading an AQE plan

[Section to be written.]

6.8 Summary

[Section to be written.]

Exercises

1. Disable AQE and run a skewed join; record the straggler task. Re-enable it and compare the plan and the task-time distribution.

Chapter 7

Partitioning, Bucketing, and Pruning

The cheapest data to move is data you never read. Partition pruning skips whole directories; dynamic partition pruning does it using values discovered from the other side of a join; bucketing lays data out so a join needs no shuffle at all. Each of these is easy to defeat — a function on the partition column, a `CAST`, a stray `SELECT *` — and the plan tells you when you have. This chapter covers each mechanism and how to confirm it is working.

7.1 Partition pruning and the pushdown killers

[Section to be written.]

7.2 Dynamic partition pruning

[Section to be written.]

7.3 Bucketing to avoid a shuffle

[Section to be written.]

7.4 repartition versus coalesce

[Section to be written.]

7.5 Column pruning and ReadSchema

[Section to be written.]

7.6 Summary

[Section to be written.]

Exercises

1. Write the same filter three ways — a function on the partition column, a CAST, and a plain literal — and check which keeps PartitionFilters populated in the scan node.

Chapter 8

Caching and Reuse

Caching keeps a result Spark would otherwise recompute — but the wrong cache wastes storage memory and starves execution, and a cache used once is pure overhead. Spark also reuses work you did not ask it to: an identical exchange or subquery computed once and read twice. This chapter covers when caching pays, which storage level to pick against the GC budget from Chapter 3, and how to see reuse in a plan. It is also where execution and storage memory finally meet at runtime.

8.1 When caching pays

[Section to be written.]

8.2 Lazy cache() versus eager CACHE TABLE

[Section to be written.]

8.3 Storage levels and the GC trade-off

[Section to be written.]

8.4 How execution and storage memory borrow at run-time

[Section to be written.]

8.5 Exchange and subquery reuse

[Section to be written.]

8.6 Unpersist and cache thrashing

[Section to be written.]

8.7 Summary

[Section to be written.]

Exercises

1. Build a query that reads one CTE twice. Show the plan reusing the exchange, then change the query so the reuse breaks and measure the difference.

Part III

The Iceberg Lakehouse

Chapter 9

Table Formats and the Iceberg Spec

An Iceberg table is not a directory of Parquet files with a manifest bolted on. It is a tree: a catalog pointer to a metadata file, to a manifest list, to manifests, to data and delete files — every level immutable, every commit a single atomic pointer swap. Reads consult this tree before touching any data. This chapter walks the spec (format version 2) against a real table's metadata tables so you can draw the tree from memory.

9.1 From Hive tables to table formats

[Section to be written.]

9.2 The Iceberg spec, format version 2

[Section to be written.]

9.3 Snapshots and the metadata tree

[Section to be written.]

9.4 Position deletes versus equality deletes

[Section to be written.]

9.5 Iceberg, Delta Lake, and Hudi

[Section to be written.]

9.6 Summary

[Section to be written.]

Exercises

1. Commit three appends to a table, then inspect `metadata_log_entries`, `snapshots`, and `manifests`. Explain how many metadata files and manifests exist and why.

Chapter 10

Catalogs and Commits

The catalog is the one component that turns a table name into the location of its current metadata — and the point at which a commit becomes atomic. Writing new metadata is not a commit until the catalog's pointer is swapped to it. When two writers race, one wins the swap and the other retries. This chapter covers what the Hive Metastore and REST catalogs do, how the atomic swap works, and how to tune retries so a busy table does not fail commits under contention.

10.1 What a catalog does

[Section to be written.]

10.2 The Hive Metastore catalog

[Section to be written.]

10.3 The REST catalog

[Section to be written.]

10.4 The atomic pointer swap

[Section to be written.]

10.5 Optimistic concurrency and retries

[Section to be written.]

10.6 Catalog configuration in Spark

[Section to be written.]

10.7 Summary

[Section to be written.]

Exercises

1. Run two writers into the same table. Observe the retry, then find the winning and losing commit attempts in the snapshot log.

Chapter 11

Iceberg Partitioning and Layout

Iceberg partitioning is hidden: you filter on `event_ts` and Iceberg derives the predicate on `day(event_ts)` for you, so a query never references a partition column directly and pruning does not break when the scheme changes. This chapter covers the partition transforms, evolving a spec without rewriting history, sort orders and Z-ordering, target file size, and the write-distribution mode — the choices that decide how much a reader and a writer have to move.

11.1 Hidden partitioning and partition transforms

[Section to be written.]

11.2 Partition evolution

[Section to be written.]

11.3 Sort orders and Z-ordering

[Section to be written.]

11.4 Target file size and the small-files problem

[Section to be written.]

11.5 Write distribution modes

[Section to be written.]

11.6 Summary

[Section to be written.]

Exercises

1. Given a daily data volume and a query pattern, choose a partition spec. Justify it against the target of 128 MB–1 GB per partition per write.

Chapter 12

Reading Iceberg Efficiently

Before the Iceberg reader opens a single data file, it has already skipped most of them — by partition, by manifest, by column statistics, and optionally by bloom filter. Each layer needs the query’s filter in a form it understands, and the scan report tells you how well each one worked. This chapter covers the pruning layers, the metrics modes that feed them, and how to read a scan report as a diagnostic.

12.1 Manifest pruning and metadata filtering

[Section to be written.]

12.2 Column statistics and predicate pushdown into Iceberg

[Section to be written.]

12.3 Metrics modes: full, truncate, counts, none

[Section to be written.]

12.4 Bloom filters

[Section to be written.]

12.5 Vectorized reads

[Section to be written.]

12.6 Split planning

[Section to be written.]

12.7 The metadata tables as a diagnostic

[Section to be written.]

12.8 Summary

[Section to be written.]

Exercises

1. Run a selective query and, from the scan report, compute the fraction of data pruned at each layer (partition, manifest, file).

Chapter 13

Writing and Maintaining Iceberg

Copy-on-write rewrites whole files on every update: slow writes, fast reads. Merge-on-read writes small delete files instead: fast writes, slower reads until compaction catches up. The choice is a function of your update rate and your read latency budget, and it commits you to a maintenance schedule — compaction, snapshot expiration, orphan-file cleanup — in a specific order. This chapter covers the write modes, optimizing MERGE, and running maintenance without deleting data a reader still needs.

13.1 Copy-on-write versus merge-on-read

[Section to be written.]

13.2 MERGE, UPDATE, and DELETE

[Section to be written.]

13.3 Write distribution and skew on write

[Section to be written.]

13.4 Compacting data files

[Section to be written.]

13.5 Rewriting manifests

[Section to be written.]

13.6 Snapshot expiration and orphan files

[Section to be written.]

13.7 Metadata management for streaming tables

[Section to be written.]

13.8 Summary

[Section to be written.]

Exercises

1. Run a MERGE with and without the partition column in the ON clause. Explain the difference in files scanned from the plan.

Part IV

Operating Spark

Chapter 14

Tuning Resources and Configuration

This chapter does not hand you a table of recommended settings. It derives an executor shape from the memory map in Chapter 3 and the shuffle in Chapter 4, so that every number — cores, heap, overhead, partition count — has a reason you can defend. It then covers dynamic allocation, the external shuffle service on Kubernetes, configuration precedence, and running Spark behind Kyuubi for many users. For the standalone list of tuning knobs with values, see `spark_memory_tuning.md` in the repository root.

14.1 Executor sizing from the mechanism

[Section to be written.]

14.2 Dynamic allocation

[Section to be written.]

14.3 Parallelism defaults

[Section to be written.]

14.4 The external and push-based shuffle service

[Section to be written.]

14.5 Configuration precedence and profiles

[Section to be written.]

14.6 Kyuubi and multi-tenancy

[Section to be written.]

14.7 Summary

[Section to be written.]

Exercises

1. Take a job that spills. Propose two different fixes — more memory versus fewer cores — and predict which the mechanism favours, and why.

Chapter 15

Debugging and Profiling

A slow job leaves evidence: in the SQL tab, the stages tab, the executor metrics, the event log, and — when you need it — a flame graph. This chapter teaches you to read that evidence in order, so that "the job is slow" becomes "stage 4 is skewed" or "the executors are GC-bound" in a few minutes. It also catalogs the failure signatures — OOM and container kills, fetch failures, stragglers — and what each looks like before you reach for a fix.

15.1 The Spark UI as a diagnostic

[Section to be written.]

15.2 Event logs and the history server

[Section to be written.]

15.3 Reading task, shuffle, spill, and GC metrics

[Section to be written.]

15.4 Common failure signatures

[Section to be written.]

15.5 async-profiler and flame graphs

[Section to be written.]

15.6 Debugging query plans

[Section to be written.]

15.7 Summary

[Section to be written.]

Exercises

1. Given three anonymized stage summaries, diagnose each as skew, under-partitioned, or GC-bound, and name the evidence.

Chapter 16

Streaming into the Lakehouse

Structured Streaming is the same query engine over an unbounded table, processing one micro-batch at a time and ending each batch with a single Iceberg commit. The whole model from the earlier parts still applies — the shuffle, the executor budget, the commit — but with two new pressures: state that must be kept between batches, and a commit cadence that produces small files faster than a batch job. This chapter covers the streaming model and what it does to an Iceberg table.

16.1 The micro-batch model and the unbounded table

[Section to be written.]

16.2 Sources and sinks

[Section to be written.]

16.3 Event time and watermarking

[Section to be written.]

16.4 Stateful operations and the state store

[Section to be written.]

16.5 Output modes and triggers

[Section to be written.]

16.6 Exactly-once and the commit

[Section to be written.]

16.7 Writing to Iceberg: cadence, small files, maintenance

[Section to be written.]

16.8 Summary

[Section to be written.]

Exercises

1. Given a trigger interval and an input rate, estimate files committed per hour and the compaction cadence needed to keep the table healthy.

Part V

Internals

Chapter 17

Tungsten and Code Generation

Below the plan, a stage does not interpret operators row by row. Tungsten stores rows as packed off-heap bytes, and whole-stage code generation collapses an operator subtree into a single generated Java method that the JIT compiles to tight machine code. This chapter opens that layer: the binary row format, off-heap memory management, cache-aware computation, and the generated code you can actually read with `EXPLAIN CODEGEN`.

17.1 The Tungsten binary row format

[Section to be written.]

17.2 Off-heap memory management

[Section to be written.]

17.3 Cache-aware computation

[Section to be written.]

17.4 Whole-stage code generation internals

[Section to be written.]

17.5 Expression code generation

[Section to be written.]

17.6 Columnar processing and vectorization

[Section to be written.]

17.7 Summary

[Section to be written.]

Exercises

1. Run `EXPLAIN CODEGEN` on a filter-and-project query. Identify which operators were fused into one generated method and which were not.

Chapter 18

The Shuffle Framework and the Scheduler

Chapter 4 treated the shuffle as a wall. This chapter builds the wall: the shuffle manager and its writers, the map output tracker that tells reduce tasks where to fetch from, the DAG scheduler that cuts a plan into stages, the task scheduler that places tasks on executors, and the block transfer service that moves the bytes. It is the machinery behind every stage boundary and every straggler.

18.1 The shuffle manager and shuffle writers

[Section to be written.]

18.2 The map output tracker

[Section to be written.]

18.3 The DAG scheduler and the task scheduler

[Section to be written.]

18.4 RPC and block transfer

[Section to be written.]

18.5 Locality and speculation

[Section to be written.]

18.6 Summary

[Section to be written.]

Exercises

1. Trace one wide query from `collect()` to task completion, naming each scheduler component it passes through.

Chapter 19

DataSource V2 and Extensions

Iceberg is not built into Spark. It plugs in through the DataSource V2 API, and the planner negotiates with it: which filters can you apply, which columns do you need, can you do this aggregate yourself? Understanding that negotiation explains why some predicates prune and others do not. This chapter covers the DSv2 read and write API, the pushdown negotiation, how Iceberg reports splits and statistics, and how to add your own Catalyst rules and SQL extensions.

19.1 The DataSource V2 read and write API

[Section to be written.]

19.2 Pushdown negotiation

[Section to be written.]

19.3 How Iceberg reports splits and statistics

[Section to be written.]

19.4 Custom Catalyst rules and SQL extensions

[Section to be written.]

19.5 Plugins

[Section to be written.]

19.6 Summary

[Section to be written.]

Exercises

1. Write a query whose filter is pushed into Iceberg and one whose filter is not. Explain the difference from the DSv2 pushdown rules.

Appendix A

The Reference Environment and the Running Example

[Section to be written.]

Appendix B

Configuration Reference

[Section to be written.]

Appendix C

Reading Query Plans

[Section to be written.]

Appendix D

Glossary

[Section to be written.]

Abbreviations

Abbreviation	Full form
AQE	Adaptive Query Execution
CBO	Cost-Based Optimization
CoW / MoR	Copy-on-Write / Merge-on-Read
DAG	Directed Acyclic Graph
DPP	Dynamic Partition Pruning
DSv2	DataSource V2 API
GC	Garbage Collection
GCS	Google Cloud Storage
GKE	Google Kubernetes Engine
HMS	Hive Metastore
JIT	Just-In-Time (compilation)
JVM	Java Virtual Machine
OLAP	Online Analytical Processing
OLTP	Online Transaction Processing
RDD	Resilient Distributed Dataset
RPC	Remote Procedure Call
SMJ	Sort-Merge Join
SQL	Structured Query Language
UDF	User-Defined Function

Technologies

Technology	What it is
Apache Spark	a distributed compute engine
Catalyst	Spark SQL's extensible query optimizer
Tungsten	Spark's binary in-memory execution back end
Apache Iceberg	an open table format for analytic datasets on object storage
Apache Parquet	a columnar file format
Hive Metastore	a catalog service mapping table names to metadata
Apache Kyuubi	a multi-tenant SQL gateway in front of Spark
Google Cloud Storage	an S3-compatible object storage system
GKE	Google's managed Kubernetes; the cluster manager here
async-profiler	a low-overhead JVM sampling profiler

Appendix E

Further Reading

[Section to be written.]

References

- [1] Apache Iceberg. *Apache Iceberg Documentation*. <https://iceberg.apache.org/docs/latest/>. 2024.
- [2] Apache Iceberg. *Apache Iceberg Table Spec*. <https://iceberg.apache.org/spec/>. Format version 2. 2024.
- [3] Apache Kyuubi. *Apache Kyuubi Documentation*. <https://kyuubi.apache.org/docs/latest/>. 2024.
- [4] Apache Spark. *Spark SQL, DataFrames and Datasets Guide (Spark 3.5.2)*. <https://spark.apache.org/docs/3.5.2/sql-programming-guide.html>. 2024.
- [5] Apache Spark. *Tuning Spark (Spark 3.5.2)*. <https://spark.apache.org/docs/3.5.2/tuning.html>. 2024.
- [6] Michael Armbrust et al. “Lakehouse: A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics”. In: *Proceedings of the 11th Conference on Innovative Data Systems Research (CIDR)*. 2021.
- [7] Michael Armbrust et al. “Spark SQL: Relational Data Processing in Spark”. In: *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*. ACM, 2015, pp. 1383–1394.
- [8] Michael Armbrust et al. “Structured Streaming: A Declarative API for Real-Time Applications in Apache Spark”. In: *Proceedings of the 2018 ACM SIGMOD International Conference on Management of Data*. ACM, 2018, pp. 601–613.
- [9] Bill Chambers and Matei Zaharia. *Spark: The Definitive Guide*. O’Reilly Media, 2018.

- [10] Jules S. Damji et al. *Learning Spark: Lightning-Fast Data Analytics*. 2nd ed. O'Reilly Media, 2020.
- [11] Holden Karau, Adi Polak, and Rachel Warren. *High Performance Spark: Best Practices for Scaling and Optimizing Apache Spark*. 2nd ed. O'Reilly Media, 2026.
- [12] Martin Kleppmann. *Designing Data-Intensive Applications*. O'Reilly Media, 2017.
- [13] Tomer Shiran, Jason Hughes, and Alex Merced. *Apache Iceberg: The Definitive Guide*. O'Reilly Media, 2024.
- [14] Matei Zaharia et al. "Apache Spark: A Unified Engine for Big Data Processing". In: *Communications of the ACM* 59.11 (2016), pp. 56–65.
- [15] Matei Zaharia et al. "Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing". In: *Proceedings of the 9th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. USENIX Association, 2012, pp. 15–28.